ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH

(CO5085)

EXERCISES E1-E3

Softmax, MLP, CNN, Transformer và LSTM/GRU trên CIFAR-10

Lớp: Lớp L01

Giảng viên: Tiến sĩ Lê Thành Sách

Học viên thực hiện: Nguyễn Trọng Anh — 2670251

TP. Hồ Chí Minh, Tháng 11/2026

Mục lục

1 Tổng quan **3**

2 Thiết lập chung **4**
2.1 Tập dữ liệu và phân tích ban đầu 4
2.2 Chia dữ liệu 5
2.3 Tiền xử lý và tăng cường dữ liệu 5
2.4 Giao thức huấn luyện và đánh giá 6
2.5 Các độ đo 6
2.6 Môi trường và khả năng tái lập 7

3 Excercise 1 **7**
3.1 Mục tiêu 7
3.2 Mô hình 7
3.3 Huấn luyện 8
3.4 Kết quả 9
3.4.1 Độ chính xác 9
3.4.2 Chi phí 9
3.4.3 Đường cong học 9
3.4.4 Phân tích lỗi 10
3.5 Thảo luận 12
3.6 Hạn chế 13

4 Excercise 2 **14**
4.1 Mục tiêu 14
4.2 Cài đặt Multi-Head Self-Attention 14
4.3 Kiểm chứng tính đúng đắn 14
4.4 Các cách tạo token 15
4.5 Mô hình và huấn luyện 16
4.6 Kết quả 17
4.6.1 Độ chính xác 17
4.6.2 Chi phí 18
4.6.3 Phân tích lỗi 19
4.6.4 Khảo sát kích thước mô hình 19
4.7 Thảo luận 19
4.8 Hạn chế 20

5 Excercise 3 **20**
5.1 Mục tiêu 20
5.2 Biểu diễn ảnh thành chuỗi 21
5.3 Mô hình và huấn luyện 21
5.4 Kết quả 22
5.4.1 Độ chính xác 22
5.4.2 Chi phí 23
5.4.3 Phụ thuộc theo thứ tự 23
5.4.4 Phân tích lỗi 25
5.5 Thảo luận 25

5.6 Hạn chế 25

6 Kết luận **26**
6.1 Tổng kết 26
6.2 Hạn chế chung 27
6.3 Hướng tiếp theo 28

1 Tổng quan

Báo cáo này trình bày các bài tập E1–E3 của môn Học sâu và ứng dụng trong thị giác máy tính (CO5085, học kỳ 261). Cả ba bài dùng chung tập CIFAR-10 và chung một cách chia dữ liệu (Mục 2.2), nên kết quả giữa các bài có thể so sánh trực tiếp.

- **E1** (Mục 3): so sánh ba bộ phân loại là softmax trên vector flatten, MLP và CNN; mỗi mô hình được huấn luyện cả khi có và không có tăng cường dữ liệu.
- **E2** (Mục 4): tự cài đặt Multi-Head Self-Attention (MSA) và đối chiếu với `nn.MultiheadAttention` của PyTorch trong một ViT nông, đồng thời so sánh ba cách tạo token từ ảnh: patch 4×4, hàng (row) và CNN-stem.
- **E3** (Mục 5): mô hình hóa ảnh như một chuỗi (theo hàng, theo cột, hoặc theo patch 4×4) và phân loại bằng LSTM và GRU; phân tích mức độ mô hình phụ thuộc vào thứ tự các bước và so sánh với MLP, CNN của E1.

Kết quả chính. Các con số dưới đây là độ chính xác trên tập test, dạng trung bình $\pm$ độ lệch chuẩn trên 5 seed.

- CNN đạt $83.58 \pm 0.25\%$ khi không tăng cường dữ liệu và $87.75 \pm 0.16\%$ khi có tăng cường, vượt xa MLP ($53.60 \pm 0.41\%$) và softmax ($37.36 \pm 0.07\%$), dù có ít tham số hơn MLP khoảng 6 lần.
- Tăng cường dữ liệu chỉ có lợi cho CNN (tăng 4.17 điểm phần trăm). Với MLP và softmax, vốn làm việc trên vector flatten và không bất biến với phép dịch chuyển, tăng cường dữ liệu làm giảm độ chính xác (còn 49.84% và 35.28%).
- MSA tự cài đặt và MSA của PyTorch cho độ chính xác tương đương ở cả ba cách tạo token: chênh lệch trung bình không quá 0.44 điểm, cùng cỡ với độ lệch chuẩn giữa các seed, và đổi dấu tùy seed. Thời gian một bước huấn luyện gần như bằng nhau, nhưng bản PyTorch dùng ít bộ nhớ hơn và suy luận nhanh hơn 12–32%.
- Trong E2, CNN-stem cho kết quả tốt nhất ($85.42 \pm 0.33\%$ với MSA tự cài đặt), tiếp đến là patch ($80.37 \pm 0.59\%$) và row ($71.85 \pm 0.30\%$). ViT tốt nhất vẫn thấp hơn CNN của E1 khi cả hai cùng dùng tăng cường dữ liệu (87.75%), dù hai bài dùng recipe tối ưu khác nhau (Mục 4).
- Trong E3, GRU tốt hơn LSTM ở cả ba cách biểu diễn dù ít tham số hơn khoảng 25%; tốt nhất là GRU trên chuỗi patch ($75.66 \pm 0.42\%$). Mọi RNN đều thua CNN của E1 (12–20 điểm) nhưng hơn MLP của E1 (14–22 điểm). Khi đảo ngược hoặc xáo trộn thứ tự các bước ở lúc test, độ chính xác với chuỗi hàng và patch sụt còn 24–31%, cho thấy RNN học được một trình tự có hướng.

Mã nguồn và tái lập. Toàn bộ mã nguồn, cấu hình và log huấn luyện nằm trong repository `https://github.com/anhtr-nguyn/co5085-dl-cv` (thư mục `E1/`, `E2/` và `E3/`); trang tổng hợp của nhóm: `https://anhtr-nguyn.github.io/co5085-dl-cv/`.

2 Thiết lập chung

Mục này mô tả những thiết lập dùng chung cho mọi thí nghiệm. Các chương sau chỉ nêu những điểm khác biệt, chẳng hạn kiến trúc và siêu tham số tối ưu.

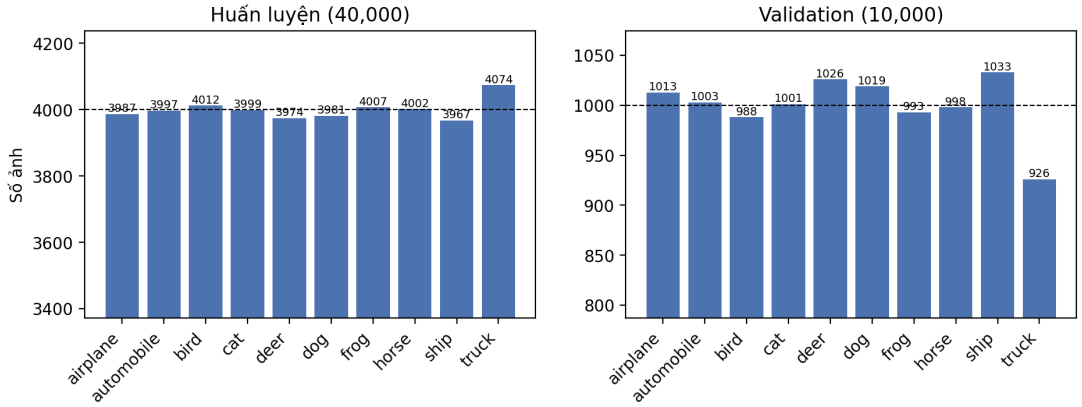
2.1 Tập dữ liệu và phân tích ban đầu

CIFAR-10 được lựa chọn cho bài tập này với: 60,000 ảnh màu kích thước 32×32, thuộc 10 lớp (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), gồm 50,000 ảnh huấn luyện và 10,000 ảnh test; mỗi lớp có số ảnh bằng nhau trong bộ gốc. CIFAR-10 được lựa chọn vì đây là ảnh màu với nội dung đa dạng hơn chữ số hay ảnh thời trang, nên sự khác biệt giữa bộ phân loại tuyến tính, MLP, CNN và Transformer thể hiện rõ hơn, trong khi ảnh vẫn đủ nhỏ để huấn luyện nhiều cấu hình với nhiều seed.

Hình 1 minh họa một số ảnh mẫu và Hình 2 cho thấy phân bố lớp của các tập sau khi chia. Ở độ phân giải 32×32, nhiều ảnh chỉ còn hình dáng và màu sắc thô; vật thể có thể bị cắt, nhỏ hoặc nằm lệch, và nền (bầu trời, mặt nước, cỏ) chiếm phần lớn ảnh ở một số lớp.



Hình 1: *Tám ảnh ngẫu nhiên của mỗi lớp trong tập huấn luyện (ảnh gốc, chưa chuẩn hóa hay tăng cường).*



Hình 2: *Số ảnh mỗi lớp trong tập huấn luyện và tập validation (đường đứt nét: mức chia đều 4,000 và 1,000).*

2.2 Chia dữ liệu

Tập test là 10,000 ảnh kiểm tra chính thức của CIFAR-10. Từ 50,000 ảnh huấn luyện gốc, chúng tôi tách 10,000 ảnh làm tập validation bằng một hoán vị ngẫu nhiên của các chỉ số, sinh bởi `numpy.random.RandomState` với seed chia dữ liệu cố định bằng 912; 10,000 chỉ số đầu của hoán vị là validation, phần còn lại là huấn luyện (Bảng 1). Cách chia này không phân tầng nên số ảnh mỗi lớp chỉ xấp xỉ đều: từ 3,967 đến 4,074 ở tập huấn luyện và từ 926 đến 1,033 ở tập validation. E1, E2 và E3 dùng chung một hàm chia dữ liệu (tệp `data.py` giống hệt nhau ở ba thư mục) với cùng seed, nên các chỉ số được sinh lại y hệt ở mỗi lần chạy, và mọi mô hình ở mọi bài được huấn luyện và đánh giá trên đúng cùng một tập ảnh. Các chỉ số này cũng được lưu ra tệp `split.npz` (`python data.py`) để đối chiếu.

Bảng 1: *Chia dữ liệu CIFAR-10 (seed chia dữ liệu = 912).*

Tập	Nguồn	Số ảnh
Huấn luyện	phần còn lại của 50,000 ảnh huấn luyện gốc	40,000
Validation	tách ngẫu nhiên từ 50,000 ảnh huấn luyện gốc	10,000
Test	10,000 ảnh kiểm tra chính thức	10,000

2.3 Tiền xử lý và tăng cường dữ liệu

Mọi ảnh được chuyển sang tensor trong khoảng $[0, 1]$ rồi chuẩn hóa theo từng kênh với trung bình $(0.491, 0.482, 0.446)$ và độ lệch chuẩn $(0.247, 0.243, 0.261)$, là các thống kê thường dùng của tập huấn luyện CIFAR-10.

Khi bật tăng cường dữ liệu, ảnh huấn luyện được áp `RandomCrop(32, padding=4)` và `RandomHorizontalFlip` trước bước chuyển sang tensor và chuẩn hóa. Tập validation và test không bao giờ được tăng cường. Để bảo đảm điều này, chúng tôi tạo hai đối tượng dataset riêng với hai bộ biến đổi khác nhau, rồi dùng chung một tập chỉ số để chọn mẫu, thay vì chia một dataset duy nhất (khi đó tập validation sẽ bị tăng cường theo). Bảng 2 nêu cấu hình tăng cường ở từng bài.

Bảng 2: *Cấu hình tăng cường dữ liệu theo bài.*

Bài	Tăng cường dữ liệu
E1	Mỗi mô hình chạy hai lượt: không tăng cường và RandomCrop + Flip
E2	RandomCrop + Flip
E3	RandomCrop + Flip

2.4 Giao thức huấn luyện và đánh giá

Theo yêu cầu của đề, vòng lặp huấn luyện được tự viết bằng PyTorch (forward → loss → backward → optimizer step), không dùng các API huấn luyện cấp cao. Sau mỗi epoch, mô hình được đánh giá trên tập validation, và checkpoint có độ chính xác validation cao nhất được lưu lại. Khi huấn luyện xong, checkpoint tốt nhất được nạp lại và đánh giá trên tập test đúng một lần. Tập test không được dùng để chọn epoch, chọn mô hình hay điều chỉnh siêu tham số; mọi lựa chọn đó chỉ dựa trên tập validation.

Vì checkpoint được chọn theo validation nên độ chính xác validation hơi lạc quan; con số được báo cáo là độ chính xác test. Mỗi cấu hình được chạy với 5 seed khởi tạo (912, 42, 100, 200, 300), trong khi seed chia dữ liệu luôn cố định ở 912. Hai seed này đóng vai trò độc lập: seed chia dữ liệu quyết định ảnh nào thuộc tập nào và không đổi giữa các lần chạy, còn seed khởi tạo quyết định trọng số ban đầu và thứ tự xáo trộn dữ liệu. Kết quả được báo cáo dưới dạng trung bình ± độ lệch chuẩn trên 5 seed.

2.5 Các độ đo

Từ ma trận nhầm lẫn $C \in \mathbb{N}^{K\times K}$ ($K = 10$), trong đó C_{ij} là số mẫu có nhãn thật i được dự đoán là j, chúng tôi tính

$$\text{acc} = \frac{\sum_i C_{ii}}{\sum_{i,j} C_{ij}}, \qquad P_c = \frac{C_{cc}}{\sum_i C_{ic}}, \qquad R_c = \frac{C_{cc}}{\sum_j C_{cj}}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{1}$$

Precision, Recall và F1 được báo cáo ở dạng macro, tức trung bình không trọng số trên K lớp. Do các lớp gần như cân bằng nên accuracy và macro-recall xấp xỉ nhau.

Về chi phí, chúng tôi báo cáo:

- **Số tham số** khả huấn luyện (`requires_grad=True`).
- **Bộ nhớ GPU đỉnh khi huấn luyện**, đọc bằng `torch.cuda.max_memory_allocated` trong log của từng run; con số này gồm tham số, gradient, trạng thái optimizer và activation của một batch 128 ảnh.
- **Tốc độ**, đo riêng bằng các script `bench.py` (E1, E2, E3) thay vì lấy từ thời gian huấn luyện trong log. Lý do là nhiều run được chạy song song trên cùng GPU, và với các mô hình nhỏ này, thời gian một epoch chủ yếu phụ thuộc vào DataLoader (đọc và tăng cường ảnh trên CPU), nên không phản ánh chi phí tính toán của mô hình. Mỗi script chạy tuần tự trên một GPU A100, batch 128 ảnh ngẫu nhiên đã nằm sẵn trên GPU, float32; sau 20 vòng khởi động, đo trung bình 100 vòng với `torch.cuda.synchronize()` trước và sau. *Thời gian huấn luyện* là thời gian của một bước forward + backward + optimizer step (ms/batch); *tốc độ inference* là số ảnh mỗi giây ở chế độ `eval()` với `torch.no_grad()`.

Mọi mô hình đều được huấn luyện đúng 100 epoch, trừ hai run khảo sát kích thước ở E2 (200 epoch).

2.6 Môi trường và khả năng tái lập

Bảng 3: *Môi trường thực nghiệm.*

Thành phần	Chi tiết
GPU	2× NVIDIA A100 80GB PCIe
CPU / RAM	AMD EPYC 7313P (16 nhân, 32 luồng) / 251 GiB
Hệ điều hành	Ubuntu 22.04.5 LTS
Python	3.10.22
PyTorch / CUDA	2.11.0 / 12.8

Mỗi lần chạy được lưu trong một thư mục riêng, không ghi đè lần chạy cũ, gồm tệp cấu hình đã áp mọi ghi đè từ dòng lệnh (`config.yaml`), log đầy đủ theo từng epoch (`log.json`), checkpoint tốt nhất, dự đoán trên tập validation và kết quả trên tập test (`test_metrics.json`). Cả ba bài đặt seed cho Python, NumPy và PyTorch, tắt `cudnn.benchmark` và bật `torch.use_deterministic_algorithms(True)`, nên cùng một seed trên cùng phần cứng và phiên bản thư viện cho cùng kết quả; trên phần cứng hoặc phiên bản khác, kết quả vẫn có thể lệch nhẹ. Độ lệch chuẩn trong mọi bảng là độ lệch chuẩn mẫu (`ddof=1`) trên 5 seed.

3 Excercise 1

3.1 Mục tiêu

E1 xây dựng và so sánh ba bộ phân loại ảnh trên CIFAR-10: (i) bộ phân loại softmax trên vector flatten, (ii) MLP trên vector flatten và (iii) CNN. Câu hỏi chính là: khi cùng dữ liệu và cùng quy trình huấn luyện, cấu trúc mô hình (tuyến tính, kết nối đầy đủ, hay tích chập) ảnh hưởng thế nào đến độ chính xác, chi phí và kiểu lỗi; và tăng cường dữ liệu tác động khác nhau ra sao lên từng loại mô hình.

3.2 Mô hình

Cả ba mô hình nhận đầu vào $x \in \mathbb{R}^{3\times32\times32}$ đã chuẩn hóa và trả về vector logits $z \in \mathbb{R}^{10}$. Hàm softmax không nằm trong mô hình mà được tính bên trong hàm mất mát cross-entropy, nhờ đó ổn định số học hơn.

- **Softmax.** Ảnh được flatten thành vector 3072 chiều rồi đi qua một lớp tuyến tính: $p(y \mid x) = \mathrm{softmax}(Wx + b)$, với $W \in \mathbb{R}^{10\times3072}$. Đây là mô hình tuyến tính nên ranh giới quyết định giữa các lớp là các siêu phẳng trong không gian pixel.
- **MLP.** Vector flatten đi qua bốn lớp ẩn kích thước $2048 \to 1024 \to 512 \to 256$ với hàm kích hoạt ReLU, rồi lớp đầu ra 10 chiều. Không dùng dropout.

- **CNN.** Bốn khối *Conv* 3×3 *(padding 1, có bias)* $\rightarrow$ *BatchNorm* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* 2×2 với số kênh $64, 128, 256, 512$, rút ảnh từ 32×32 về 2×2; tiếp theo là global average pooling và một lớp tuyến tính $512 \rightarrow 10$.

Bảng 4 tóm tắt kiến trúc và số tham số. Đáng chú ý, CNN có ít tham số hơn MLP khoảng 5.8 lần.

Bảng 4: *Kiến trúc và số tham số khả huấn luyện của ba mô hình E1.*

Mô hình	Kiến trúc	Tham số
Softmax	Flatten $\rightarrow$ Linear$(3072, 10)$	30,730
MLP	Flatten $\rightarrow$ $[2048, 1024, 512, 256]$ (ReLU) $\rightarrow$ 10	9,050,378
CNN	$4\times$[Conv3×3–BN–ReLU–MaxPool], kênh 64/128/256/512 $\rightarrow$ GAP $\rightarrow$ 10	1,558,026

3.3 Huấn luyện

Cả ba mô hình dùng chung một quy trình huấn luyện (Bảng 5) để so sánh công bằng: khác biệt về kết quả chỉ đến từ kiến trúc và từ việc có hay không tăng cường dữ liệu. Mỗi mô hình được huấn luyện hai lượt (có và không tăng cường), mỗi lượt với 5 seed khởi tạo, tổng cộng $3\times2\times5 = 30$ lần chạy. Vòng lặp huấn luyện được tự viết (Thuật toán 1).

Bảng 5: *Quy trình huấn luyện dùng chung cho E1.*

Thành phần	Giá trị
Hàm mất mát	Cross-entropy
Optimizer	Adam, learning rate 10^{-3}, weight decay 0
Lịch learning rate	Không (learning rate cố định)
Số epoch	100
Batch size	128
Tăng cường dữ liệu	Không, hoặc RandomCrop + Flip (Mục 2.3)
Seed khởi tạo	912, 42, 100, 200, 300
Chọn checkpoint	Độ chính xác validation cao nhất

for *mỗi batch* (x, y) *của tập huấn luyện* **do**
  $z \leftarrow f_\theta(x)$; `// forward, ra logits`
  $\mathcal{L} \leftarrow \mathrm{CE}(z, y)$;
  đặt gradient về 0; tính $\nabla_\theta \mathcal{L}$ bằng backward;
  cập nhật θ bằng một bước Adam;
end
đánh giá trên validation; nếu độ chính xác cao nhất từ trước đến nay thì lưu checkpoint;

Algorithm 1: Một epoch huấn luyện tự viết.

3.4 Kết quả

3.4.1 Độ chính xác

Bảng 6 cho thấy thứ hạng CNN > MLP > softmax được giữ nguyên ở cả hai cấu hình. Độ lệch chuẩn giữa các seed đều nhỏ (không quá 0.41 điểm phần trăm), trong khi khoảng cách giữa các mô hình lên đến hàng chục điểm, nên thứ hạng này không phụ thuộc vào seed.

Bảng 6: *Độ chính xác trên tập test (%, trung bình ± độ lệch chuẩn trên 5 seed).*

Mô hình	Tham số	Không tăng cường	Có tăng cường	Chênh lệch
Softmax	30,730	37.36 ± 0.07	35.28 ± 0.11	-2.08
MLP	9,050,378	53.60 ± 0.41	49.84 ± 0.30	-3.76
CNN	1,558,026	$\mathbf{83.58 \pm 0.25}$	$\mathbf{87.75 \pm 0.16}$	$+4.17$

3.4.2 Chi phí

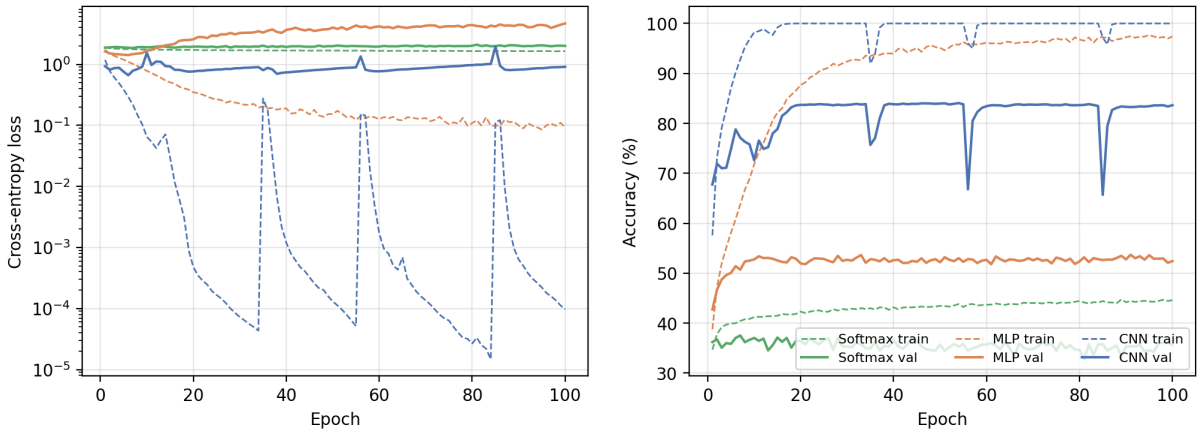
Bảng 7: *Chi phí và macro-F1 trên tập test (%, trung bình ± độ lệch chuẩn trên 5 seed, không tăng cường). Thời gian một bước huấn luyện và tốc độ inference đo bằng* `E1/bench.py` *(batch 128, A100, float32; Mục 2.5); bộ nhớ đỉnh lấy từ log huấn luyện.*

Mô hình	Tham số	Huấn luyện (ms/batch)	Inference (ảnh/s)	Bộ nhớ đỉnh (MB)	Macro-F1 test
Softmax	30,730	0.59	4,699,120	19	36.67 ± 0.09
MLP	9,050,378	1.85	393,121	192	53.63 ± 0.47
CNN	1,558,026	3.80	178,440	212	$\mathbf{83.54 \pm 0.26}$

CNN có ít tham số hơn MLP 5.8 lần nhưng một bước huấn luyện chậm hơn khoảng 2 lần (3.80 so với 1.85 ms) và tốc độ inference thấp hơn 2.2 lần, vì mỗi bộ lọc tích chập được áp lên mọi vị trí không gian: số phép tính phụ thuộc vào kích thước bản đồ đặc trưng chứ không chỉ vào số tham số. Bộ nhớ đỉnh của hai mô hình xấp xỉ nhau nhưng vì lý do khác nhau: với MLP, phần lớn là tham số, gradient và hai moment của Adam (khoảng 4×4 byte cho mỗi tham số, tức khoảng 145 MB cho 9 triệu tham số), còn với CNN phần lớn là activation của các bản đồ đặc trưng cần giữ lại cho backward. Trong thực tế, các mô hình này nhỏ đến mức thời gian một epoch do DataLoader quyết định: các run softmax và CNN không tăng cường (chạy tuần tự) đều mất khoảng 2.1–2.2 s mỗi epoch, dù một bước huấn luyện của hai mô hình chênh nhau khoảng 6 lần.

3.4.3 Đường cong học

Hình 3 trình bày loss và độ chính xác trên tập huấn luyện và validation theo epoch.



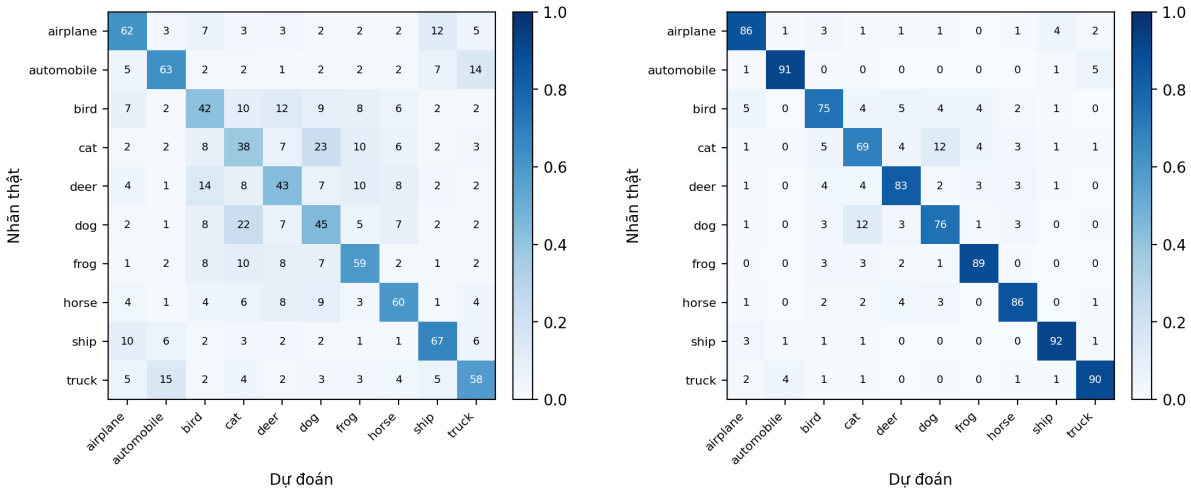
Hình 3: *Loss (trái, thang log) và độ chính xác (phải) trên tập huấn luyện (nét đứt) và validation (nét liền) của ba mô hình, cấu hình không tăng cường, seed 912.*

Ba mô hình cho ba kiểu đường cong khác nhau (số liệu của seed 912):

- **Softmax** underfit: train acc chỉ đạt 44.60% sau 100 epoch, val acc cao nhất 37.53% ngay ở epoch 7 rồi giảm nhẹ về 35.65%, val loss thấp nhất cũng ở epoch 7. Mô hình tuyến tính không đủ năng lực để khớp cả tập huấn luyện, và sau vài epoch chỉ còn khớp thêm nhiễu.
- **MLP** overfit rõ: train acc tăng lên 97.41% trong khi val acc dừng quanh 52–54% từ khoảng epoch 10. Val loss thấp nhất ở epoch 6 (1.415) rồi tăng đều lên 4.664 ở epoch 100, tức mô hình ngày càng tự tin vào các dự đoán sai. Vì val acc gần như phẳng, epoch tốt nhất dao động rất rộng giữa các seed (từ 15 đến 92).
- **CNN** cũng học thuộc tập huấn luyện (train acc đạt 99.9% từ epoch 17), nhưng val acc vẫn giữ ở mức 83–84%; khoảng cách train–val khoảng 16 điểm. Val loss thấp nhất ở epoch 6 (0.662) rồi tăng lên 0.908 mà val acc không giảm, nên ở đây val loss tăng chủ yếu do mô hình quá tự tin chứ không phải do sai nhiều hơn. Đường cong còn có các đợt mất ổn định: train loss bất ngờ tăng vọt và val acc tụt xuống còn khoảng 66% (epoch 56 và 85), rồi hồi phục sau vài epoch. Hiện tượng này xuất hiện ở cả 5 seed khi không tăng cường (val acc tụt hơn 5 điểm trong một epoch, 1–3 lần mỗi run) và không xuất hiện ở run nào có tăng cường; nhiều khả năng do Adam với learning rate cố định 10^{-3} khi loss đã gần bằng 0. Nhờ chọn checkpoint theo val acc, các đợt này không ảnh hưởng đến kết quả test.

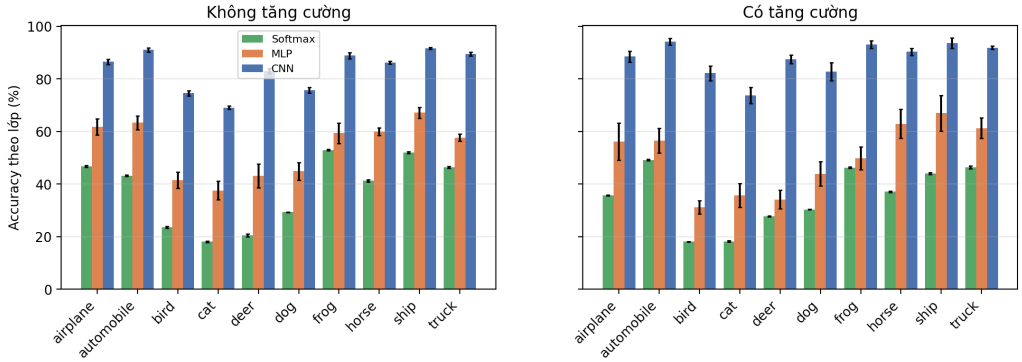
3.4.4 Phân tích lỗi

Hình 4 so sánh ma trận nhầm lẫn của MLP và CNN; Hình 5 cho độ chính xác theo từng lớp; Hình 6 minh họa một số mẫu bị CNN phân loại sai. Ma trận nhầm lẫn và độ chính xác theo lớp được tính trên tập test, cộng dồn hoặc lấy trung bình trên 5 seed; các mẫu lỗi lấy từ tập validation, để không phải nhìn vào ảnh test khi phân tích.

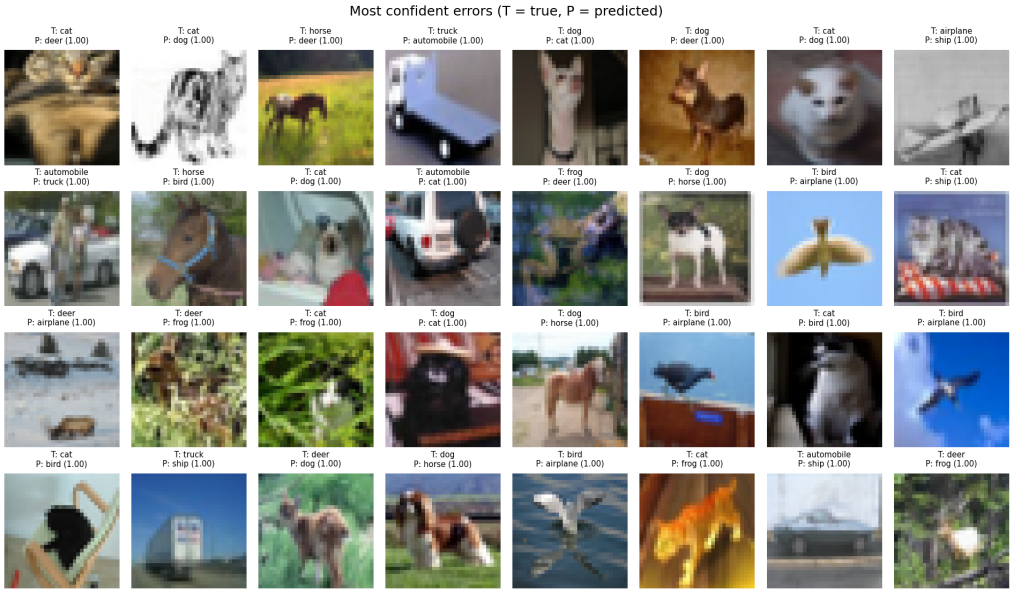


(a) *MLP* **(b)** *CNN*

Hình 4: *Ma trận nhầm lẫn chuẩn hóa theo hàng (%, mỗi hàng là một lớp thật) trên tập test, cấu hình không tăng cường, cộng dồn trên 5 seed.*



Hình 5: *Độ chính xác theo lớp (recall) của ba mô hình trên tập test, khi không và có tăng cường dữ liệu (trung bình ± độ lệch chuẩn trên 5 seed).*



Hình 6: *32 ảnh validation mà CNN (không tăng cường, seed 912) phân loại sai với xác suất cao nhất, kèm nhãn thật (T), nhãn dự đoán (P) và xác suất của lớp dự đoán.*

Khi gộp cả hai chiều nhầm lẫn, cặp lớp bị nhầm nhiều nhất ở cả ba mô hình (không tăng cường) là *cat* và *dog*: 11.9% số ảnh của hai lớp này bị CNN gán sang lớp còn lại, 22.4% với MLP và 18.3% với softmax. Hai lớp này có hình dạng, tư thế và bối cảnh rất giống nhau ở độ phân giải 32×32, nên ngay cả CNN cũng khó phân biệt. Các cặp tiếp theo của CNN nhỏ hơn hẳn: automobile–truck (4.7%), bird–deer (4.5%) và bird–cat (4.4%). Với CNN, cat là lớp có độ chính xác thấp nhất (69.1%) và ship cao nhất (91.6%); khi có tăng cường, cat tăng lên 73.7% nhưng vẫn thấp nhất, còn automobile cao nhất (94.2%). Với softmax, nhầm lẫn lớn nhất theo một chiều không phải cat–dog mà là deer $\rightarrow$ frog (27.2% số ảnh deer), cùng với automobile–truck (17.9%) và airplane–ship (16.2%).

Các mẫu lỗi trong Hình 6 đều được dự đoán với xác suất làm tròn bằng 1.00, tức khi không tăng cường, CNN sai một cách rất tự tin, phù hợp với val loss tăng ở Hình 3. Phần lớn lỗi thuộc ba nhóm: (i) bị nền chi phối, ví dụ chim trên nền trời xanh bị đoán là airplane, xe tải trên nền xanh bị đoán là ship, mèo hoặc hươu trong cỏ bị đoán là frog; (ii) góc chụp hoặc bố cục lạ, như ảnh cận mặt, vật thể bị cắt hoặc chiếm rất ít điểm ảnh; (iii) các cặp vốn giống nhau như cat–dog, automobile–truck, deer–horse, trong đó một số ảnh khó phân biệt ngay cả bằng mắt ở 32×32.

3.5 Thảo luận

Số tham số không quyết định độ chính xác. CNN có ít tham số hơn MLP khoảng 5.8 lần nhưng chính xác hơn khoảng 30 điểm phần trăm khi không tăng cường dữ liệu (83.58% so với 53.60%). Lý do nằm ở cấu trúc: mỗi bộ lọc tích chập dùng chung trọng số trên mọi vị trí và chỉ nhìn một vùng lân cận nhỏ, nên CNN mặc định giả sử rằng một đặc trưng (cạnh, vân, bộ phận vật thể) có ý nghĩa như nhau ở mọi nơi trong ảnh. Giả thiết này đúng với ảnh tự nhiên. MLP thì phải học riêng từng pixel ở từng vị trí, và việc flatten xóa bỏ thông tin về lân cận không gian; phần lớn tham số của nó (khoảng 6.3 triệu, ở lớp đầu tiên) được dùng để học những quan hệ mà CNN có sẵn trong cấu trúc. Softmax là mô hình tuyến tính trên pixel nên chỉ đạt 37.36%, thấp hơn MLP khoảng 16

điểm, cho thấy các lớp ẩn phi tuyến giúp ích nhưng không bù được việc thiếu cấu trúc không gian.

Tăng cường dữ liệu chỉ có lợi cho CNN. Tăng cường dữ liệu tăng độ chính xác của CNN thêm 4.17 điểm, nhưng làm giảm độ chính xác của MLP (-3.76) và softmax (-2.08). `RandomCrop` dịch ảnh tối đa 4 pixel. Với CNN, tích chập và pooling vốn gần như bất biến với phép dịch nhỏ, nên các ảnh bị dịch vẫn cho đặc trưng tương tự; tăng cường dữ liệu khi đó hoạt động như một bộ điều chuẩn và giảm overfit. Với mô hình trên vector flatten, cùng một vật thể bị dịch đi vài pixel lại là một vector hoàn toàn khác, nên mô hình phải dùng năng lực hữu hạn để học thuộc nhiều phiên bản của cùng một ảnh, trong khi ảnh test không bị dịch. Kết quả này cho thấy tác dụng của tăng cường dữ liệu phụ thuộc vào việc mô hình có khai thác được tính bất biến mà phép biến đổi tạo ra hay không. Độ chính xác trên tập huấn luyện ở epoch cuối (trung bình 5 seed) xác nhận điều này. Với CNN, train acc chỉ giảm nhẹ từ 99.90% xuống 98.48%, còn khoảng cách giữa train acc và val acc tốt nhất thu hẹp từ khoảng 16 xuống khoảng 10 điểm, tức tăng cường dữ liệu giảm overfit mà gần như không làm bài toán khó hơn với mô hình. Với MLP, train acc giảm mạnh từ 97.50% xuống 69.21%, và với softmax từ 44.58% xuống 30.04% (thấp hơn cả val acc 35.84%): hai mô hình này không còn khớp được tập huấn luyện đã tăng cường, nên cả train và val đều giảm.

Kiểu lỗi. Việc cặp cat–dog đứng đầu ở cả ba mô hình cho thấy đây là độ khó nội tại của dữ liệu ở độ phân giải thấp, chứ không chỉ là điểm yếu của một kiến trúc. Ngoài cặp này, lỗi của MLP và softmax phân tán hơn CNN: 5 ô nhầm lẫn lớn nhất chiếm 18.9% tổng số lỗi của MLP và 17.2% của softmax, so với 23.7% của CNN. Các cặp nhầm lớn của hai mô hình flatten thường có màu nền giống nhau, như airplane–ship (nền trời và nền nước, 10.8% với MLP, 16.2% với softmax), bird–deer và deer–frog (nền cỏ, cây). Điều này phù hợp với việc mô hình trên vector flatten dựa nhiều vào phân bố màu toàn cục hơn là vào hình dạng của vật thể.

3.6 Hạn chế

- Cả ba mô hình dùng chung một quy trình huấn luyện (Adam, 10^{-3}, 100 epoch) mà chưa tinh chỉnh riêng. Một MLP được điều chuẩn tốt hơn (dropout, weight decay, dừng sớm) có thể thu hẹp phần nào khoảng cách, dù khó vượt CNN.
- Độ lệch chuẩn được tính trên 5 seed khởi tạo với một cách chia dữ liệu cố định, nên chỉ phản ánh dao động do khởi tạo và thứ tự dữ liệu, không phản ánh dao động do cách chia.
- Chỉ thử một loại tăng cường (RandomCrop + Flip) và một tập dữ liệu; kết luận về tăng cường dữ liệu chưa được kiểm tra trên các phép biến đổi khác.

4 Excercise 2

4.1 Mục tiêu

E2 tự cài đặt Multi-Head Self-Attention (MSA) từ các phép toán cơ bản, đối chiếu với `nn.MultiheadAttention` của PyTorch, rồi đặt MSA vào một bộ phân loại ảnh dạng Vision Transformer (ViT) nông trên CIFAR-10. Hai câu hỏi chính là: (i) bản tự cài đặt có tương đương bản PyTorch về kết quả và chi phí không; (ii) cách biến ảnh thành chuỗi token ảnh hưởng thế nào đến độ chính xác và chi phí.

4.2 Cài đặt Multi-Head Self-Attention

Với chuỗi đầu vào $X \in \mathbb{R}^{N\times C}$ gồm N token, mỗi token C chiều, và h head có kích thước $d = C/h$, MSA tính

$$[Q, K, V] = XW_{qkv} + b_{qkv}, \qquad W_{qkv} \in \mathbb{R}^{C\times 3C}, \tag{2}$$

$$A_i = \mathrm{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d}}\right) \in \mathbb{R}^{N\times N}, \qquad i = 1, \ldots, h, \tag{3}$$

$$\mathrm{MSA}(X) = \left[A_1 V_1;\ \ldots;\ A_h V_h\right] W_o + b_o, \qquad W_o \in \mathbb{R}^{C\times C}, \tag{4}$$

trong đó $Q_i, K_i, V_i \in \mathbb{R}^{N\times d}$ là phần của head thứ i và $[\cdot;\cdot]$ là phép ghép theo chiều đặc trưng. Hệ số $1/\sqrt{d}$ giữ cho phương sai của tích vô hướng không tăng theo d, tránh việc softmax bị bão hòa. Softmax được lấy theo chiều key, nên mỗi hàng của A_i có tổng bằng 1.

Bản tự cài đặt (`MyMSA`) chỉ dùng `nn.Linear`, phép nhân ma trận và `softmax`. Một lớp tuyến tính duy nhất sinh đồng thời Q, K, V; tensor kết quả được đổi hình theo chuỗi $[B, N, 3C] \to [B, N, 3, h, d] \to [3, B, h, N, d]$, ma trận attention có kích thước $[B, h, N, N]$, và đầu ra các head được ghép lại thành $[B, N, C]$ trước phép chiếu W_o. Bản đối chiếu (`TorchMSA`) bọc `nn.MultiheadAttention` với `batch_first=True` và `need_weights=False`. Để hai bản bắt đầu từ cùng một phân phối khởi tạo, `MyMSA` khởi tạo W_{qkv} bằng Xavier uniform và đặt b_{qkv}, b_o bằng 0, giống cách `nn.MultiheadAttention` khởi tạo. Cả hai bản chỉ áp dropout lên ma trận attention (tham số `dropout` của `nn.MultiheadAttention`), không có dropout sau phép chiếu W_o.

4.3 Kiểm chứng tính đúng đắn

Trước khi huấn luyện, chúng tôi kiểm chứng bản tự cài đặt bằng các phép thử trong Bảng 8. Phép thử quan trọng nhất là sao chép trọng số: trọng số của `nn.MultiheadAttention` (`in_proj_weight`, `in_proj_bias`, `out_proj`) được chép vào `MyMSA`, rồi so sánh đầu ra trên cùng một đầu vào (CPU, float32, chế độ `eval`). Phép thử được thực hiện cả ở mức một lớp MSA và ở mức toàn bộ mô hình.

Bảng 8: *Các phép thử kiểm chứng MSA tự cài đặt.*

Phép thử	Kết quả
Số tham số `MyMSA` và `TorchMSA` ($C = 128$, $h = 4$)	Bằng nhau: $4C^2 + 4C = 66{,}048$
Khớp đầu ra một lớp MSA khi chung trọng số	Sai lệch tuyệt đối lớn nhất 1.4×10^{-7}
Khớp đầu ra toàn mô hình khi chung trọng số	Sai lệch tuyệt đối lớn nhất 9.5×10^{-7} (patch), 8.3×10^{-7} (row), 6.0×10^{-7} (CNN-stem)
Khớp gradient toàn mô hình khi chung trọng số	Sai lệch tương đối lớn nhất trên mọi tham số 1.0×10^{-6} (patch), 9.3×10^{-7} (row), 9.9×10^{-7} (CNN-stem)
Hoán vị thứ tự token, không positional embedding	Logits gần như không đổi (sai lệch 1.5×10^{-7})
Hoán vị thứ tự token, có positional embedding	Logits thay đổi (sai lệch 4.0×10^{-2})
Học thuộc một batch 32 mẫu nhãn ngẫu nhiên (AdamW, 10^{-3}, 200 bước, không dropout)	Loss từ khoảng 2.3–2.5 giảm về dưới 0.01 ở 5/6 tổ hợp (0.0012–0.0087); riêng CNN-stem + PyTorch còn 0.63 sau 200 bước. Khi hai bản CNN-stem dùng chung trọng số khởi tạo và chạy 400 bước, cả hai đều về dưới 0.01 (0.0026 và 0.0041)

Sai lệch cỡ 10^{-7} là mức sai số làm tròn của float32, nên hai bản tính cùng một hàm, cả ở chiều forward lẫn backward. Trong phép thử học thuộc, CNN-stem với MSA PyTorch là tổ hợp duy nhất chưa về gần 0 sau 200 bước. Đây không phải lỗi cài đặt: khi hai bản CNN-stem bắt đầu từ cùng trọng số, cả hai đều còn loss khoảng 2.0–2.1 ở bước 200 và cùng về dưới 0.01 ở bước 400. CNN-stem đơn giản là hội tụ chậm hơn trong phép thử này, và tốc độ hội tụ phụ thuộc vào khởi tạo. Hai phép thử hoán vị (thực hiện không có token CLS, gộp bằng trung bình các token) minh họa một tính chất quan trọng của self-attention: khi không có positional embedding, MSA hoán vị đầu ra theo đúng cách hoán vị đầu vào, nên sau bước gộp mô hình hoàn toàn không biết token nào nằm ở đâu trong ảnh. Positional embedding là thành phần duy nhất cung cấp thông tin vị trí cho mô hình.

4.4 Các cách tạo token

Chúng tôi so sánh ba cách biến ảnh $3 \times 32 \times 32$ thành chuỗi token $C = 128$ chiều (Hình 7, Bảng 9):

- **Patch 4×4.** Ảnh được chia thành lưới 8×8 ô không chồng lấn; mỗi ô ($3 \times 4 \times 4 = 48$ giá trị) được chiếu tuyến tính thành một token, cài đặt bằng một Conv 4×4 với stride 4. Kết quả là 64 token.
- **Hàng (row).** Mỗi hàng ảnh ($3 \times 32 = 96$ giá trị) được chiếu tuyến tính thành một token, cho 32 token. Toàn bộ cấu trúc theo chiều ngang nằm trong một phép chiếu tuyến tính duy nhất; attention chỉ mô hình hóa quan hệ giữa các hàng.

- **CNN-stem.** Hai khối Conv $3 \times 3 \to$ ReLU $\to$ MaxPool 3×3 (stride 2), mỗi khối 128 kênh, không có BatchNorm, rút ảnh về bản đồ đặc trưng 8×8; mỗi vị trí không gian là một token, cho 64 token. Mỗi token nhìn một vùng 13×13 pixel, và hai token kề nhau cách nhau 4 pixel, nên vùng nhìn của chúng chồng lấn.



Hình 7: *Minh họa ba cách tạo token trên một ảnh test. Khung đỏ là vùng ảnh tạo nên một token; với CNN-stem, khung đỏ và vàng là vùng nhìn* 13×13 *của hai token kề nhau.*

Bảng 9: *Số token và số tham số của ba bộ tạo token* ($C = 128$).

Bộ tạo token	Số token	Tham số	Đặc điểm
Patch 4×4	64	6,272	Chiếu tuyến tính từng ô, không chồng lấn
Hàng	32	12,416	Chiếu tuyến tính cả hàng
CNN-stem	64	151,168	Hai lớp tích chập, vùng nhìn chồng lấn

Cần lưu ý rằng CNN-stem có nhiều tham số hơn bộ tạo token patch khoảng 24 lần và tốn nhiều phép tính hơn, vì tích chập được tính trên bản đồ đặc trưng 32×32 và 16×16 trước khi rút về 8×8.

4.5 Mô hình và huấn luyện

Mô hình là một ViT nông: bộ tạo token, token CLS học được ghép vào đầu chuỗi, positional embedding học được cộng vào mọi token, 4 khối encoder Pre-LN, một LayerNorm cuối, rồi lớp tuyến tính trên token CLS để ra 10 logits. Mỗi khối encoder tính

$$X' = X + \mathrm{MSA}(\mathrm{LN}(X)), \tag{5}$$

$$X'' = X' + \mathrm{FFN}(\mathrm{LN}(X')), \qquad \mathrm{FFN}(u) = W_2\,\mathrm{GELU}(W_1 u + b_1) + b_2, \tag{6}$$

với $C = 128$, $h = 4$ head và lớp ẩn FFN 256 chiều (tỉ lệ 2). Hai loại MSA được hoán đổi mà không thay đổi bất kỳ thành phần nào khác, nên với cùng bộ tạo token, hai bản có đúng cùng số tham số (Bảng 11). Mỗi khối có 132,480 tham số; bốn khối chiếm 529,920 tham số, phần còn lại thuộc về bộ tạo token, token CLS, positional embedding và lớp phân loại.

Quy trình huấn luyện (Bảng 10) khác E1 ở optimizer và lịch learning rate, vì ViT nhỏ huấn luyện từ đầu thường kém ổn định nếu bắt đầu ngay với learning rate cao. Mọi tổ hợp bộ tạo token $\times$ loại MSA dùng chung quy trình này, với 5 seed khởi tạo, tổng cộng $3 \times 2 \times 5 = 30$ lần chạy.

Bảng 10: *Quy trình huấn luyện dùng chung cho E2 (`E2/config/tinyvit.yaml`).*

Thành phần	Giá trị
Hàm mất mát	Cross-entropy
Optimizer	AdamW, learning rate 10^{-3}, weight decay 0.05 (không áp cho bias, LayerNorm, token CLS, positional embedding)
Lịch learning rate	Warmup tuyến tính 5 epoch, sau đó cosine về 0; cập nhật theo từng iteration
Gradient clipping	Chuẩn L_2 tối đa 1.0
Dropout	0.1 trên ma trận attention, trong FFN và sau khi cộng positional embedding
Số epoch / batch size	100 / 128
Tăng cường dữ liệu	RandomCrop + Flip (Mục 2.3)
Seed khởi tạo	912, 42, 100, 200, 300
Chọn checkpoint	Độ chính xác validation cao nhất

4.6 Kết quả

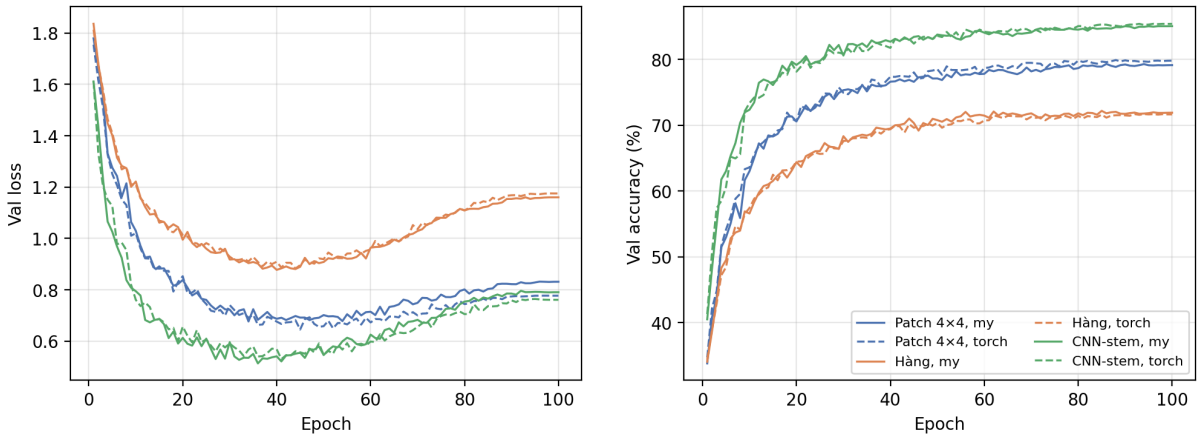
4.6.1 Độ chính xác

Bảng 11 tổng hợp kết quả của 6 tổ hợp. Với cả ba bộ tạo token, hai loại MSA cho kết quả gần như trùng nhau, và Hình 8 cho thấy đường cong validation của hai bản cũng chồng lên nhau trong suốt quá trình huấn luyện.

Bảng 11: *Kết quả E2 (%, trung bình ± độ lệch chuẩn trên 5 seed, có tăng cường dữ liệu). Bộ nhớ đỉnh lấy từ log huấn luyện.*

Bộ tạo token	MSA	Tham số	Val acc	Test acc	Macro-F1 test	Bộ nhớ đỉnh (MB)
Patch 4×4	Tự cài đặt	546,186	79.81 ± 0.52	80.37 ± 0.59	80.30 ± 0.62	324
	PyTorch	546,186	79.84 ± 0.25	79.93 ± 0.28	79.88 ± 0.25	267
Hàng	Tự cài đặt	548,234	72.09 ± 0.42	71.85 ± 0.30	71.86 ± 0.31	163
	PyTorch	548,234	71.76 ± 0.67	72.00 ± 0.51	72.04 ± 0.52	152
CNN-stem	Tự cài đặt	691,082	85.52 ± 0.38	85.42 ± 0.33	85.39 ± 0.32	463
	PyTorch	691,082	85.48 ± 0.43	$\mathbf{85.47 \pm 0.21}$	$\mathbf{85.43 \pm 0.21}$	406

So sánh theo từng seed (cùng seed thì cùng thứ tự dữ liệu), hiệu test acc giữa bản tự cài đặt và bản PyTorch nằm trong khoảng -0.22 đến $+1.18$ điểm với patch (trung bình $+0.44$), -0.70 đến $+0.69$ với hàng (trung bình -0.14) và -0.46 đến $+0.51$ với CNN-stem (trung bình -0.05). Hiệu trung bình cùng cỡ với độ lệch chuẩn giữa các seed và đổi dấu tùy seed, nên không có bản nào tốt hơn một cách nhất quán. Điều này khớp với kiểm chứng ở Mục 4.3: hai bản tính cùng một hàm, chỉ khác ở thứ tự phép toán dấu phẩy động, và sai lệch nhỏ này đủ để hai lần huấn luyện đi theo hai quỹ đạo khác nhau.



Hình 8: *Val loss (trái) và val accuracy (phải) theo epoch của 6 tổ hợp, seed 912. Nét liền: MSA tự cài đặt; nét đứt: MSA PyTorch.*

Về bộ tạo token, thứ hạng CNN-stem > patch > hàng rõ ràng và giống nhau với cả hai loại MSA: CNN-stem hơn patch khoảng 5 điểm, patch hơn hàng khoảng 8 điểm, lớn hơn nhiều so với độ lệch chuẩn. Ở cả 6 tổ hợp, val loss thấp nhất rơi vào khoảng epoch 29–54 rồi tăng dần, trong khi val acc vẫn tăng chậm đến cuối (epoch tốt nhất từ 66 đến 100): mô hình tiếp tục sửa được thêm một ít mẫu nhưng ngày càng quá tự tin với các mẫu sai, giống hiện tượng ở E1. Ở epoch cuối, train acc là khoảng 94.5% với patch, 93.3% với hàng và 99.3% với CNN-stem; khoảng cách train–val lớn nhất ở bộ tạo token hàng (khoảng 21 điểm, so với khoảng 15 điểm của patch và 14 điểm của CNN-stem).

4.6.2 Chi phí

Bảng 12 so sánh tốc độ, đo riêng bằng `E2/bench.py` theo cách mô tả ở Mục 2.5. Thời gian huấn luyện theo epoch trong log không được dùng vì các run được chạy song song trên cùng GPU.

Bảng 12: *Tốc độ của 6 tổ hợp (A100, batch 128, float32). Số token chưa tính token CLS.*

Bộ tạo token	MSA	Số token	Huấn luyện (ms/batch)	Inference (ảnh/s)
Patch 4×4	Tự cài đặt	64	10.36	59,179
	PyTorch	64	10.43	69,251
Hàng	Tự cài đặt	32	10.37	80,564
	PyTorch	32	10.54	106,644
CNN-stem	Tự cài đặt	64	11.08	47,902
	PyTorch	64	10.71	53,591

Thời gian một bước huấn luyện của hai bản MSA chênh nhau không quá 3.5%, theo cả hai chiều. Khi suy luận, bản PyTorch nhanh hơn 17% với patch, 32% với hàng và 12% với CNN-stem, và khi huấn luyện dùng ít bộ nhớ hơn 11–57 MB (Bảng 11). Lý do là với `need_weights=False`, `nn.MultiheadAttention` gọi `scaled_dot_product_attention`, một kernel hợp nhất không cần lưu ma trận attention $[B, h, N, N]$ ra bộ nhớ. Bản tự

cài đặt thì tạo ra ma trận này cùng kết quả softmax và dropout của nó, rồi giữ lại cho backward. Phần bộ nhớ tiết kiệm được lớn hơn khi chuỗi dài hơn (57 MB với $N = 65$ so với 11 MB với $N = 33$), vì ma trận attention tăng theo N^2.

Giữa các bộ tạo token, CNN-stem tốn nhất: so với patch, nó có thêm khoảng 145 nghìn tham số (+27%), bộ nhớ đỉnh khi huấn luyện tăng từ 324 lên 463 MB và tốc độ inference giảm khoảng 19% (bản tự cài đặt), vì hai lớp tích chập được tính trên bản đồ đặc trưng 32×32 và 16×16. Bộ tạo token hàng rẻ nhất nhờ chuỗi chỉ dài một nửa, nhưng cũng kém chính xác nhất.

4.6.3 Phân tích lỗi

Ở cả 6 tổ hợp, cặp nhầm lẫn lớn nhất vẫn là cat–dog, và cat là lớp có F1 thấp nhất: khoảng 0.54 với hàng, 0.64 với patch và 0.73 với CNN-stem (trung bình 5 seed trên tập test). Bộ tạo token hàng nhầm automobile–truck nhiều hơn hẳn các bộ khác (khoảng 9–10% số ảnh automobile bị đoán là truck, so với khoảng 6–7% ở patch và dưới 5% ở CNN-stem). Patch đặc biệt hay nhầm deer sang horse (khoảng 7% số ảnh deer). Như ở E1, lớp tốt nhất là automobile (patch, CNN-stem) hoặc ship (hàng).

4.6.4 Khảo sát kích thước mô hình

Để xem khoảng cách giữa patch và CNN-stem có phải chỉ do số tham số hay không, chúng tôi chạy thêm hai mô hình patch lớn hơn với MSA PyTorch, mỗi mô hình một lần (seed 912) và 200 epoch (Bảng 13).

Bảng 13: *Khảo sát kích thước với bộ tạo token patch, MSA PyTorch (%). Hai dòng cuối chỉ có 1 seed và 200 epoch.*

Cấu hình	Epoch	Tham số	Val acc	Test acc
$C = 128$, 4 khối, FFN tỉ lệ 2 (baseline, 5 seed)	100	546,186	79.84 ± 0.25	79.93 ± 0.28
$C = 128$, 6 khối, FFN tỉ lệ 2	200	811,146	80.39	80.62
$C = 256$, 4 khối, FFN tỉ lệ 4	200	3,191,562	81.80	81.91
CNN-stem, $C = 128$, 4 khối (5 seed)	100	691,082	85.48 ± 0.43	85.47 ± 0.21

Tăng số tham số gần 6 lần và gấp đôi số epoch chỉ giúp mô hình patch tăng khoảng 2 điểm, vẫn thấp hơn CNN-stem khoảng 3.5 điểm dù CNN-stem nhỏ hơn 4.6 lần. Vì hai mô hình lớn vừa được huấn luyện lâu hơn vừa chỉ có một seed, phép so sánh này chỉ mang tính định hướng; tuy vậy, mức tăng nhỏ cho thấy khoảng cách giữa các bộ tạo token chủ yếu không đến từ số tham số.

4.7 Thảo luận

MSA tự cài đặt đúng, nhưng chậm và tốn bộ nhớ hơn. Bản tự cài đặt khớp với `nn.MultiheadAttention` đến sai số làm tròn float32 ở cả forward lẫn backward, và cho cùng độ chính xác khi huấn luyện đầy đủ. Khác biệt duy nhất là hiệu năng: kernel hợp nhất của PyTorch tránh việc lưu ma trận attention, nên suy luận nhanh hơn 12–32% và dùng ít bộ nhớ hơn. Ở quy mô của bài này ($N \leq 65$), thời gian một bước huấn luyện của

hai bản gần như bằng nhau, nhưng chênh lệch về bộ nhớ sẽ tăng theo N^2 khi ảnh lớn hơn hoặc patch nhỏ hơn.

Cách tạo token quyết định thiên kiến quy nạp của ViT. Self-attention tự nó không biết gì về cấu trúc 2D của ảnh; mọi thông tin về vị trí và lân cận đến từ bộ tạo token và positional embedding. Bộ tạo token hàng gộp cả một hàng 96 giá trị vào một phép chiếu tuyến tính, nên cấu trúc theo chiều ngang chỉ được giữ ở dạng tuyến tính và attention chỉ mô hình hóa quan hệ giữa các hàng; đây là bộ kém nhất và overfit nhiều nhất. Patch 4×4 giữ được tính cục bộ theo cả hai chiều, nhưng mỗi token chỉ là phép chiếu tuyến tính của 48 pixel và các patch không chồng lấn. CNN-stem đưa vào đúng những giả thiết đã giúp CNN thắng ở E1: tính cục bộ, chia sẻ trọng số và vùng nhìn chồng lấn (13×13, bước 4), nên mỗi token đã là một đặc trưng phi tuyến của vùng lân cận. Với 40,000 ảnh huấn luyện, ViT nhỏ không đủ dữ liệu để tự học các giả thiết này từ đầu, nên đưa chúng vào qua bộ tạo token có lợi hơn nhiều so với tăng kích thước mô hình (Bảng 13).

So với CNN của E1. ViT tốt nhất (CNN-stem, 85.47%) vẫn thấp hơn CNN của E1 khi có tăng cường (87.75%), dù có ít tham số hơn (691 nghìn so với 1.56 triệu) và dùng quy trình huấn luyện được điều chỉnh cho Transformer. Hai bài khác nhau cả về kiến trúc, số tham số lẫn quy trình huấn luyện, nên khoảng cách 2.3 điểm không thể quy hết cho attention; kết quả chỉ cho thấy ở quy mô dữ liệu này, một ViT nông chưa vượt được một CNN thuần.

4.8 Hạn chế

- Cả 6 tổ hợp dùng chung một quy trình huấn luyện (Bảng 10), chưa tinh chỉnh riêng cho từng bộ tạo token. Một bộ tạo token có thể được lợi nhiều hơn từ learning rate, weight decay hay số epoch khác.
- Khảo sát kích thước chỉ có 1 seed và khác baseline cả về số epoch lẫn số tham số, nên không tách được hai yếu tố này.
- Tốc độ được đo bằng dữ liệu ngẫu nhiên trên một loại GPU, batch 128, float32; chưa đo với mixed precision, batch khác, hay số phép tính (FLOPs).
- Không thử bản không có positional embedding hay không có token CLS khi huấn luyện đầy đủ; vai trò của positional embedding mới chỉ được minh họa bằng phép thử hoán vị.

5 Excercise 3

5.1 Mục tiêu

E3 mô hình hóa mỗi ảnh CIFAR-10 như một chuỗi các bước thời gian và phân loại bằng mạng hồi quy LSTM và GRU (`nn.LSTM`, `nn.GRU`). Ba câu hỏi chính là: (i) cách cắt ảnh thành chuỗi (theo hàng, theo cột, theo patch) ảnh hưởng thế nào đến độ chính xác và chi phí; (ii) LSTM hay GRU phù hợp hơn; (iii) mô hình hồi quy có thực sự khai thác thứ tự

các bước hay không, so với các mô hình không hồi quy của E1 (MLP, CNN) trên cùng cách chia dữ liệu.

5.2 Biểu diễn ảnh thành chuỗi

Ảnh đã chuẩn hóa $x \in \mathbb{R}^{3\times32\times32}$ được đổi thành chuỗi $s \in \mathbb{R}^{T\times F}$ bằng một phép đổi hình, không có tham số (Bảng 14). Với cách theo hàng, bước thứ t là hàng ảnh thứ t (cả 3 kênh), đọc từ trên xuống; theo cột là cột thứ t, đọc từ trái sang phải; theo patch là patch 4×4 thứ t, duyệt theo hàng (trái sang phải, trên xuống dưới), giống thứ tự token của bộ tạo token patch ở E2. Code kiểm tra rằng mỗi bước lấy đúng vùng ảnh tương ứng (`python models.py`).

Bảng 14: *Ba cách biểu diễn ảnh thành chuỗi.*

Cách	Mỗi bước thời gian	T	F
Hàng	một hàng ảnh (3×32 giá trị), từ trên xuống	32	96
Cột	một cột ảnh (3×32 giá trị), từ trái sang phải	32	96
Patch	một patch 4×4 không chồng lấn ($3 \times 4 \times 4$ giá trị), theo thứ tự raster	64	48

5.3 Mô hình và huấn luyện

Chuỗi đi qua một LSTM hoặc GRU hai tầng, một chiều, kích thước ẩn 256, dropout 0.1 giữa hai tầng. Hidden state của tầng trên cùng ở bước cuối cùng, h_T, đi qua Dropout(0.1) và một lớp tuyến tính $256 \to 10$ để ra logits. Vì chỉ dùng h_T, mô hình phải tích lũy thông tin của toàn bộ ảnh qua các bước. Với LSTM, mỗi bước tính

$$[i_t, f_t, o_t] = \sigma(W_{\{i,f,o\}} s_t + U_{\{i,f,o\}} h_{t-1} + b_{\{i,f,o\}}), \qquad \tilde{c}_t = \tanh(W_c s_t + U_c h_{t-1} + b_c), \tag{7}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t), \tag{8}$$

còn GRU chỉ có hai cổng (reset, update) và không có cell state riêng, nên mỗi tầng có 3 thay vì 4 bộ trọng số. Do đó GRU có ít tham số hơn LSTM khoảng 25% (Bảng 15). Chuỗi patch có ít tham số hơn vì $F = 48$ nhỏ hơn $F = 96$, chỉ ảnh hưởng đến tầng đầu.

Bảng 15: *Số tham số khả huấn luyện của mô hình E3.*

Mô hình	Hàng / cột ($F = 96$)	Patch ($F = 48$)
LSTM	891,402	842,250
GRU	669,194	632,330

Quy trình huấn luyện (Bảng 16) theo khung của E2 (AdamW, warmup rồi cosine, gradient clipping), với weight decay và số epoch warmup nhỏ hơn. Gradient clipping đặc biệt quan trọng với mạng hồi quy, vì gradient được nhân qua nhiều bước thời gian và có thể bùng nổ. Trước khi huấn luyện đầy đủ, cả 6 tổ hợp đều học thuộc được một batch 32 mẫu nhãn ngẫu nhiên (loss từ khoảng 2.30 về dưới 0.001 sau 200 bước Adam). Tổng cộng có $2 \times 3 \times 5 = 30$ lần chạy.

Bảng 16: *Quy trình huấn luyện dùng chung cho E3 (`E3/config/rnn.yaml`).*

Thành phần	Giá trị
Hàm mất mát	Cross-entropy
Optimizer	AdamW, learning rate 10^{-3}, weight decay 0.01 (không áp cho bias)
Lịch learning rate	Warmup tuyến tính 2 epoch, sau đó cosine về 0; cập nhật theo từng iteration
Gradient clipping	Chuẩn L_2 tối đa 1.0
Số epoch / batch size	100 / 128
Tăng cường dữ liệu	RandomCrop + Flip (Mục 2.3)
Seed khởi tạo	912, 42, 100, 200, 300
Chọn checkpoint	Độ chính xác validation cao nhất

5.4 Kết quả

5.4.1 Độ chính xác

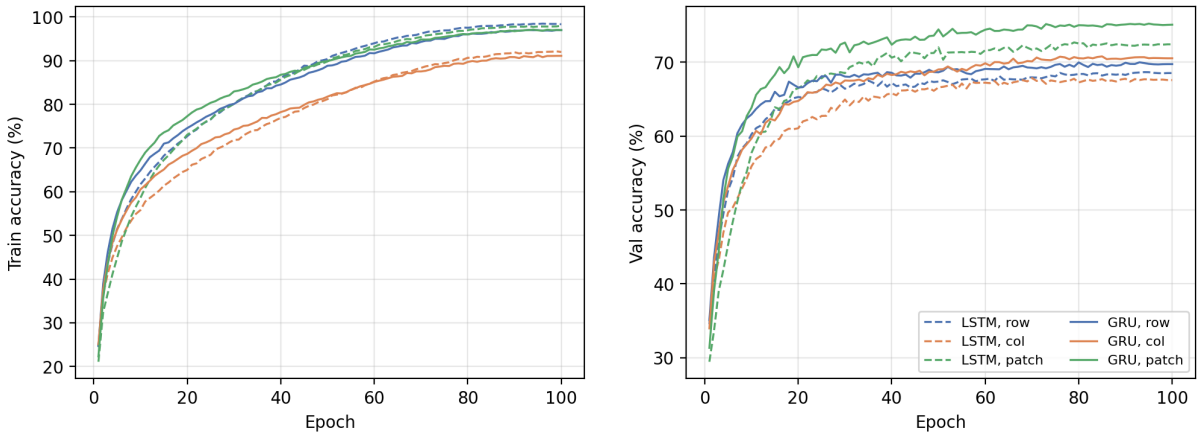
Bảng 17 tổng hợp kết quả, kèm hai baseline không hồi quy của E1 ở cấu hình tốt nhất của mỗi mô hình (MLP không tăng cường, CNN có tăng cường).

Bảng 17: *Kết quả E3 (%, trung bình ± độ lệch chuẩn trên 5 seed). Hai dòng cuối là baseline của E1 trên cùng cách chia dữ liệu.*

Mô hình	Chuỗi	Tham số	Val acc	Test acc	Macro-F1 test	Bộ nhớ đỉnh (MB)
LSTM	Hàng	891,402	68.40 ± 0.33	68.15 ± 0.59	68.09 ± 0.58	129
LSTM	Cột	891,402	67.70 ± 0.41	67.47 ± 0.09	67.48 ± 0.09	128
LSTM	Patch	842,250	72.39 ± 0.26	72.27 ± 0.31	72.26 ± 0.30	209
GRU	Hàng	669,194	69.84 ± 0.32	69.40 ± 0.31	69.37 ± 0.30	128
GRU	Cột	669,194	70.79 ± 0.18	70.90 ± 0.32	70.91 ± 0.30	126
GRU	Patch	632,330	75.61 ± 0.30	$\mathbf{75.66 \pm 0.42}$	$\mathbf{75.67 \pm 0.41}$	211
E1 MLP (không tăng cường)	–	9,050,378	53.74 ± 0.10	53.60 ± 0.41	53.63 ± 0.47	192
E1 CNN (có tăng cường)	–	1,558,026	88.34 ± 0.18	87.75 ± 0.16	87.70 ± 0.18	212

GRU tốt hơn LSTM ở cả ba cách biểu diễn: hơn 1.25 điểm với hàng, 3.43 với cột và 3.39 với patch, trong khi có ít tham số hơn khoảng 25%. Patch là cách biểu diễn tốt nhất với cả hai loại cell (GRU 75.66%, LSTM 72.27%); hàng và cột cho kết quả gần nhau (67–71%). Mọi RNN đều thua CNN của E1 từ 12 đến 20 điểm, nhưng hơn MLP của E1 từ 14 đến 22 điểm với số tham số ít hơn 10–14 lần.

Hình 9 cho thấy hai mô hình theo cột có train acc thấp hơn hẳn ở cuối quá trình huấn luyện (khoảng 91–92%, so với 97–98% của hàng và patch), nhưng val acc không kém. Như ở E1 và E2, val loss thấp nhất rơi vào khoảng epoch 19–41 rồi tăng dần trong khi val acc vẫn tăng chậm (epoch tốt nhất từ 57 đến 99). Khoảng cách train–val ở epoch cuối là 20–30 điểm, lớn hơn ViT của E2 (14–22 điểm).



Hình 9: *Train accuracy (trái) và val accuracy (phải) theo epoch của 6 tổ hợp, seed 912. Nét liền: GRU; nét đứt: LSTM.*

5.4.2 Chi phí

Bảng 18 cho tốc độ đo bằng `E3/bench.py` (Mục 2.5); thời gian theo epoch trong log không được dùng vì 6 run được chạy song song.

Bảng 18: *Tốc độ của mô hình E3 (A100, batch 128, float32), so với CNN của E1.*

Mô hình	Chuỗi	T	Huấn luyện (ms/batch)	Inference (ảnh/s)
LSTM	Hàng	32	5.65	158,065
LSTM	Cột	32	6.24	160,525
LSTM	Patch	64	9.01	96,634
GRU	Hàng	32	5.91	160,621
GRU	Cột	32	6.02	163,178
GRU	Patch	64	8.90	97,492
E1 CNN	–	–	3.80	178,440

Chi phí của RNN chủ yếu phụ thuộc vào độ dài chuỗi, vì các bước thời gian phải được tính tuần tự. Gấp đôi T (patch so với hàng, cột) làm một bước huấn luyện chậm hơn khoảng 1.5 lần, tốc độ inference giảm khoảng 40% và bộ nhớ đỉnh tăng từ khoảng 128 lên khoảng 210 MB, vì activation của mọi bước phải được giữ lại cho backward. LSTM và GRU có tốc độ gần như bằng nhau dù GRU ít tham số hơn. Ở quy mô này, cả 6 mô hình RNN đều chậm hơn và kém chính xác hơn CNN của E1.

5.4.3 Phụ thuộc theo thứ tự

Để kiểm tra mô hình có thực sự khai thác thứ tự các bước hay không, chúng tôi đánh giá mọi checkpoint tốt nhất trên tập test với ba phép biến đổi chỉ áp ở lúc test (mô hình được huấn luyện trên thứ tự gốc):

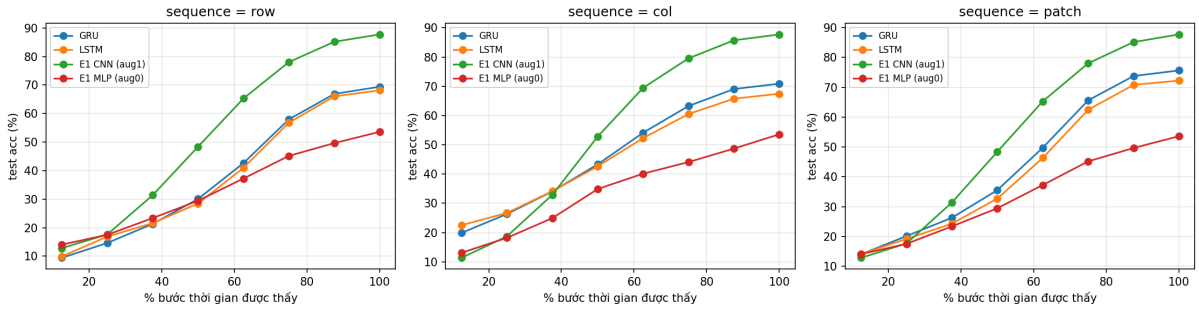
- **Prefix** $k\%$**:** RNN chỉ đọc $k\%$ bước đầu rồi phân loại từ hidden state tại đó. Với CNN và MLP, phần ảnh tương ứng với các bước đó được giữ nguyên, phần còn lại đặt bằng 0 (tức giá trị trung bình sau chuẩn hóa).

- **Đảo ngược:** đọc chuỗi từ cuối về đầu. Với hàng, phép này tương đương lật ảnh theo chiều dọc; với cột, tương đương lật ngang; với patch, tương đương đảo thứ tự các patch (xoay lưới patch 180°, giữ nguyên nội dung từng patch).
- **Xáo trộn:** một hoán vị ngẫu nhiên cố định của các bước, dùng chung cho mọi ảnh.

Bảng 19 và Hình 10 trình bày kết quả. CNN và MLP của E1 được đánh giá bằng cùng phép biến đổi trên ảnh; với patch, prefix theo thứ tự raster nên với CNN và MLP tương đương việc giữ $k\%$ số hàng trên cùng, giống cách hàng.

Bảng 19: *Độ chính xác trên tập test (%, trung bình ± độ lệch chuẩn trên 5 seed) khi chỉ thấy một phần chuỗi, khi đảo ngược và khi xáo trộn thứ tự các bước.*

Chuỗi	Mô hình	Đủ chuỗi	Prefix 25%	Prefix 50%	Prefix 75%	Đảo ngược	Xáo trộn
Hàng ($T = 32$)	GRU	69.40 ± 0.31	14.64 ± 0.60	30.08 ± 0.64	57.98 ± 0.78	30.82 ± 0.50	25.74 ± 0.86
	LSTM	68.15 ± 0.59	16.87 ± 1.07	28.49 ± 0.63	56.79 ± 0.66	30.69 ± 0.56	27.15 ± 0.93
	E1 CNN	87.75 ± 0.16	17.63 ± 1.44	48.40 ± 1.61	78.06 ± 1.30	39.47 ± 0.46	24.77 ± 2.41
	E1 MLP	53.60 ± 0.41	17.50 ± 1.22	29.44 ± 1.37	45.17 ± 0.84	25.32 ± 0.72	27.05 ± 1.22
Cột ($T = 32$)	GRU	70.90 ± 0.32	26.33 ± 0.62	43.33 ± 0.56	63.26 ± 0.26	70.50 ± 0.45	29.70 ± 0.25
	LSTM	67.47 ± 0.09	26.70 ± 0.63	42.65 ± 0.56	60.53 ± 0.46	67.60 ± 0.47	35.59 ± 0.88
	E1 CNN	87.75 ± 0.16	18.69 ± 1.47	52.84 ± 2.36	79.57 ± 0.75	88.02 ± 0.20	23.49 ± 2.60
	E1 MLP	53.60 ± 0.41	18.19 ± 0.88	34.89 ± 1.08	44.09 ± 0.36	52.36 ± 0.67	35.90 ± 0.48
Patch ($T = 64$)	GRU	75.66 ± 0.42	20.12 ± 1.03	35.51 ± 0.68	65.63 ± 0.49	29.39 ± 0.67	25.19 ± 0.87
	LSTM	72.27 ± 0.31	19.15 ± 1.51	32.69 ± 0.78	62.48 ± 0.69	29.66 ± 0.66	24.27 ± 0.77
	E1 CNN	87.75 ± 0.16	17.63 ± 1.44	48.40 ± 1.61	78.06 ± 1.30	26.74 ± 0.70	22.94 ± 0.47
	E1 MLP	53.60 ± 0.41	17.50 ± 1.22	29.44 ± 1.37	45.17 ± 0.84	24.19 ± 1.03	16.53 ± 0.67



Hình 10: *Độ chính xác trên tập test theo phần trăm số bước được thấy (trung bình 5 seed), với ba cách biểu diễn chuỗi.*

Prefix. Độ chính xác của RNN tăng dần theo số bước đã đọc, từ gần mức ngẫu nhiên ở 25% đầu chuỗi (15–27%) lên 57–66% ở 75%, và chỉ đạt mức cuối cùng khi đọc hết chuỗi: thông tin được tích lũy dần qua các bước, và phần cuối chuỗi vẫn đóng góp đáng kể. Lưu ý rằng RNN chỉ được huấn luyện để phân loại ở bước cuối, nên prefix đo lượng thông tin về lớp đã có trong hidden state ở bước trung gian mà không cần huấn luyện lại. Với chuỗi theo cột, RNN đọc 25% số cột bên trái đã đạt 26–27%, cao hơn CNN và MLP khi thấy cùng phần ảnh (18–19%). Một cách giải thích là với RNN, chuỗi ngắn hơn vẫn là một kiểu đầu vào hợp lệ, còn với CNN và MLP, ảnh bị che một phần lớn là kiểu đầu vào chưa từng gặp khi huấn luyện. Từ khoảng 50% trở đi, CNN vượt lên rõ rệt vì trích đặc trưng từ phần ảnh nhìn thấy tốt hơn hẳn.

Đảo ngược và xáo trộn. Với chuỗi hàng và patch, đảo ngược làm RNN sụt từ 68–76% còn khoảng 29–31%, và xáo trộn còn 24–27%: RNN học được một trình tự có hướng (từ trên xuống) và không bất biến với thứ tự. CNN và MLP cũng sụt mạnh (CNN còn 39.47% khi lật dọc, khoảng 23–25% khi xáo trộn), nhưng vì lý do khác: CNN dựa vào bố cục 2D cục bộ, còn mỗi trọng số của MLP gắn với một vị trí pixel cố định. Chuỗi theo cột là trường hợp ngoại lệ đáng chú ý: đảo ngược gần như không ảnh hưởng (GRU 70.90 → 70.50%, LSTM 67.47 → 67.60%), vì đảo ngược thứ tự cột chính là phép lật ngang mà `RandomHorizontalFlip` đã áp trong lúc huấn luyện. Đây là tính bất biến học được từ tăng cường dữ liệu, không phải dấu hiệu RNN bỏ qua thứ tự: khi xáo trộn các cột, độ chính xác vẫn sụt còn 30–36%. Điều này cũng giải thích train acc thấp hơn của các mô hình theo cột: mô hình phải nhận ra cùng một vật thể khi đọc theo cả hai chiều.

5.4.4 Phân tích lỗi

Như ở E1 và E2, cặp nhầm lẫn lớn nhất ở cả 6 tổ hợp là cat–dog (khoảng 15–20% số ảnh dog bị đoán là cat và ngược lại), tiếp theo là automobile–truck (khoảng 8–12% số ảnh automobile bị đoán là truck). Cat là lớp có F1 thấp nhất (0.48–0.59, trung bình 5 seed trên tập test), còn ship có F1 cao nhất (0.81–0.86) ở 5/6 tổ hợp; riêng LSTM theo cột có lớp tốt nhất là automobile (0.78).

5.5 Thảo luận

GRU tốt hơn LSTM. Với chuỗi ngắn (32–64 bước) và 40,000 ảnh huấn luyện, cơ chế cổng gọn hơn của GRU đủ để giữ thông tin qua toàn bộ chuỗi, trong khi ít tham số hơn và có lẽ dễ tối ưu hơn. Lợi thế của cell state riêng trong LSTM thường thể hiện ở chuỗi dài hơn nhiều; ở đây nó không bù được số tham số tăng thêm. Vì cả hai dùng chung một quy trình huấn luyện chưa tinh chỉnh riêng, kết luận này chỉ đúng trong thiết lập của bài.

Cách biểu diễn: patch tốt nhất. Mỗi bước của chuỗi patch là một vùng 4×4 cục bộ, nên cấu trúc 2D được giữ trong từng bước, và chuỗi dài gấp đôi cho RNN nhiều lần cập nhật hơn. Với hàng và cột, mỗi bước là cả một dải 96 giá trị, nên quan hệ không gian theo chiều còn lại chỉ được nhìn qua thứ tự thời gian. Đổi lại, patch tốn thời gian và bộ nhớ gần gấp đôi. Thứ hạng patch > hàng cũng lặp lại kết quả của ViT ở E2, nhưng RNN với patch (75.66%) vẫn thấp hơn ViT với patch (80.37%): một mô hình chỉ nhìn ảnh qua một hidden state duy nhất, đọc tuần tự, khó nối hai vùng ở xa nhau trong ảnh hơn self-attention, nơi mọi cặp token tương tác trực tiếp.

So với baseline E1. RNN nằm giữa MLP và CNN. So với MLP trên vector flatten, đọc ảnh theo chuỗi giữ được một phần cấu trúc không gian (thứ tự hàng, cột hoặc patch, và việc dùng chung trọng số giữa các bước), nên tốt hơn 14–22 điểm với số tham số ít hơn 10–14 lần. So với CNN, RNN thiếu tính cục bộ theo hai chiều và không có phân cấp đặc trưng không gian, nên vẫn thua 12–20 điểm, dù có số tham số cùng bậc.

5.6 Hạn chế

- Chỉ thử RNN một chiều, hai tầng, kích thước ẩn 256 và phân loại từ hidden state cuối; chưa thử RNN hai chiều, gộp mọi bước (mean/max pooling), hay kích thước

khác. Các lựa chọn này có thể thu hẹp khoảng cách giữa hàng, cột và patch.

- Một quy trình huấn luyện chung cho cả 6 tổ hợp, chưa tinh chỉnh riêng.
- Các phép thử prefix, đảo ngược và xáo trộn chỉ áp ở lúc test, và tăng cường dữ liệu (lật ngang) làm thay đổi kết quả của chuỗi theo cột; chưa tách được ảnh hưởng của tăng cường bằng cách huấn luyện lại không tăng cường.
- Phép thử prefix với CNN và MLP thay phần bị che bằng 0, một kiểu đầu vào chưa gặp khi huấn luyện, nên so sánh với RNN ở các mức prefix thấp không hoàn toàn công bằng.

6 Kết luận

6.1 Tổng kết

Bảng 20 tập hợp kết quả của mọi mô hình trên cùng tập test CIFAR-10 và cùng cách chia dữ liệu.

Bảng 20: *Tổng hợp độ chính xác trên tập test (%, trung bình trên 5 seed). Các mô hình E2 dùng MSA tự cài đặt; bản PyTorch cho kết quả tương đương (Bảng 11). E2 và E3 chỉ được huấn luyện với tăng cường dữ liệu.*

Bài	Mô hình	Tham số	Không tăng cường	Có tăng cường
E1	Softmax	30,730	37.36	35.28
E1	MLP	9,050,378	53.60	49.84
E1	CNN	1,558,026	83.58	**87.75**
E2	ViT, token theo hàng	548,234	–	71.85
E2	ViT, patch 4×4	546,186	–	80.37
E2	ViT, CNN-stem	691,082	–	85.42
E3	LSTM, chuỗi theo hàng	891,402	–	68.15
E3	LSTM, chuỗi theo cột	891,402	–	67.47
E3	LSTM, chuỗi patch 4×4	842,250	–	72.27
E3	GRU, chuỗi theo hàng	669,194	–	69.40
E3	GRU, chuỗi theo cột	669,194	–	70.90
E3	GRU, chuỗi patch 4×4	632,330	–	75.66

Các kết quả chính có thể tóm tắt như sau.

1. **Cấu trúc mô hình quan trọng hơn số tham số.** CNN có ít tham số hơn MLP khoảng 5.8 lần nhưng chính xác hơn khoảng 30 điểm khi không tăng cường dữ liệu và khoảng 38 điểm khi có tăng cường. Chia sẻ trọng số và vùng nhìn cục bộ của tích chập phù hợp với cấu trúc của ảnh tự nhiên, điều mà mô hình trên vector flatten phải tự học từ dữ liệu.
2. **Tác dụng của tăng cường dữ liệu phụ thuộc vào kiến trúc.** RandomCrop + Flip tăng độ chính xác của CNN thêm 4.17 điểm nhưng làm giảm độ chính xác của

MLP và softmax, vì chỉ CNN khai thác được tính bất biến với phép dịch mà phép biến đổi này tạo ra.

3. **MSA tự cài đặt là đúng và tương đương bản PyTorch về độ chính xác.** Khi chung trọng số, hai bản cho đầu ra trùng nhau đến sai số float32 (cỡ 10^{-7}); khi huấn luyện độc lập, chênh lệch độ chính xác trung bình không quá 0.44 điểm, cùng cỡ với dao động giữa các seed và đổi dấu tùy seed. Ưu thế của bản PyTorch nằm ở chi phí: ít bộ nhớ hơn và suy luận nhanh hơn 12–32% nhờ kernel attention hợp nhất.

4. **Cách tạo token quyết định phần lớn hiệu năng của ViT nhỏ.** CNN-stem (85.42%) > patch (80.37%) > hàng (71.85%). Tăng mô hình patch lên khoảng 3.2 triệu tham số (200 epoch, 1 seed) chỉ đạt 81.91%, nên lợi thế của CNN-stem đến từ cấu trúc tích chập chứ không chỉ từ số tham số.

5. **RNN đọc ảnh như chuỗi nằm giữa MLP và CNN.** GRU tốt hơn LSTM ở cả ba cách biểu diễn dù ít tham số hơn khoảng 25%, và chuỗi patch tốt nhất (GRU 75.66%). Mọi RNN hơn MLP của E1 14–22 điểm nhưng thua CNN 12–20 điểm. Khi đảo ngược hoặc xáo trộn thứ tự các bước ở lúc test, độ chính xác với chuỗi hàng và patch sụt còn 24–31%, nên RNN thực sự dựa vào thứ tự đọc; riêng chuỗi theo cột gần như bất biến với đảo ngược vì phép này trùng với lật ngang đã dùng khi tăng cường dữ liệu.

Một mạch chung xuyên suốt ba bài: trên một tập dữ liệu nhỏ như CIFAR-10 (40,000 ảnh huấn luyện), mô hình càng mang nhiều giả thiết cấu trúc phù hợp với ảnh thì càng tốt. Ở E1, CNN vượt xa các mô hình trên vector flatten; ở E2, bộ tạo token càng giống CNN thì ViT càng tốt, nhưng ViT tốt nhất (85.42%) vẫn chưa vượt CNN của E1 (87.75%). Kết quả này phù hợp với nhận định rằng Transformer cần nhiều dữ liệu hơn hoặc huấn luyện trước để bù cho việc thiếu các giả thiết cấu trúc đó. Ở E3, RNN chỉ giữ được cấu trúc không gian theo một chiều (thứ tự các bước) và nén cả ảnh vào một hidden state, nên đứng sau cả CNN lẫn ViT cùng cách cắt patch (75.66% so với 80.37%); chuỗi patch, vốn giữ được tính cục bộ 2D trong từng bước, lại là cách biểu diễn tốt nhất cho cả RNN lẫn ViT.

6.2 Hạn chế chung

- **Một tập dữ liệu, một cách chia.** Mọi kết luận chỉ được kiểm tra trên CIFAR-10 với một cách chia validation cố định; độ lệch chuẩn trên 5 seed chỉ phản ánh dao động do khởi tạo và thứ tự dữ liệu.

- **Quy trình huấn luyện chưa tinh chỉnh riêng.** Mỗi bài dùng một quy trình chung cho mọi mô hình trong bài; một số mô hình (đặc biệt MLP và ViT theo hàng) có thể cải thiện với siêu tham số và điều chuẩn phù hợp hơn.

- **So sánh giữa các bài không hoàn toàn cùng điều kiện.** E1, E2 và E3 dùng optimizer, lịch learning rate và quy mô mô hình khác nhau, nên so sánh CNN với ViT và RNN chỉ mang tính tham khảo.

- **Mô hình nhỏ và không huấn luyện trước.** Các kết luận về ViT có thể thay đổi với mô hình lớn hơn hoặc khi có trọng số huấn luyện trước.

- **Đo thời gian.** Nhiều run được huấn luyện song song trên cùng GPU, nên thời gian theo epoch trong log không dùng để so sánh. Tốc độ được đo riêng bằng `bench.py` (Mục 2.5): tuần tự trên một A100, dữ liệu ngẫu nhiên đã nằm sẵn trên GPU, batch 128, float32. Số liệu này phản ánh chi phí tính toán của mô hình chứ không phải thời gian huấn luyện thực tế, vốn chủ yếu do DataLoader quyết định với các mô hình nhỏ; nó cũng chưa xét mixed precision, batch khác hay phần cứng khác.

6.3 Hướng tiếp theo

- Với E3, thử RNN hai chiều và gộp mọi bước thời gian thay vì chỉ dùng hidden state cuối, và huấn luyện lại chuỗi theo cột không dùng lật ngang để tách ảnh hưởng của tăng cường dữ liệu lên phép thử đảo ngược.
- Tinh chỉnh siêu tham số riêng cho từng mô hình trên tập validation để so sánh công bằng hơn giữa các kiến trúc.
- Đo riêng tốc độ của hai bản MSA theo số token N để tách chi phí attention khỏi các phần khác của mô hình.
- Kiểm tra câu hỏi CNN và Transformer khi có nhiều dữ liệu hơn và có huấn luyện trước; đây chính là nội dung của bài tập lớn A1 trên Food-101.